

KICKSTARTER

For creators

Log in

 Search projects, creators, and categories

Crafts Dance Design Fashion Film Food Games Journalism Music Photography Publishing Technology Theater

Software Architecture: The Art Of Engineering Your Code



Master the art of engineering your code. Scale your effort by transforming your development approach.

Follow along!

Created by

Knight's Tour

46 backers pledged CA\$ 5,623 to help bring this project to life.**Last updated** January 4, 2020

[Campaign](#)[Rewards](#)[Creator](#)[FAQ](#)[Updates](#) ¹⁶[Comments](#) ¹⁷[Community](#)

Funding period

Nov 27 2018 - Dec 27 2018 (30 days)

Software Architecture: The Art Of Engineering Your Code

📍 [Toronto, Canada](#) 🛠️ [Software](#)

CA\$ 5,623

pledged of CA\$ 5,000 goal

46

backers

Story

This course will take you on a journey to become a Software Architect.

A Software Architect is a Software Engineer. Most developers are not skilled in Software Engineering practices and only a few software developers are Software Engineers.

Software Engineers **ARE** in demand.

If you want to belong to this **elite group** of developers and want to take your development skills to the next level, this course is for you. Please read on...



Focus on the Big Picture, NOT the Little Details

Software development is becoming very complex, and source code is becoming hard to manage. We have AI, VR, AR and a host of other development areas that are growing exponentially, as well as a myriad of ubiquitous IoT smart devices such as smart TVs, smart fridges, and drones that can follow you and take professional-grade footage of your activities, all without an operator.

We also have self-driving cars.

All run on code. And in all successful cases, **somebody designed this code.**

Consider the following statistics:

- The code for the Apollo 11 Lunar Lander was about **145,000** lines of tightly packed and very well organized code.
- The code that runs the Mars Curiosity rover is about **2,500,000** lines of code which is about 17 times more lines than the Lunar Lander.
- the Android OS is about **12,000,000** lines of code! This is about 80 times more lines of code.
- The entire Google set of Google Services comes in at about **2,000,000,000 (2 billion)** lines of code. This is about 14,000 times more code than the Lunar Lander.

This trend will continue and will accelerate. Increasingly, pieces of software are generated by other pieces of software, and there is even a strong indication that soon we will have **AI that will be writing** its own, adaptable code. So... **we must have ways of managing** this rapidly growing volume of code and the complexities that come with it.

The answer to all this lies in Software Engineering. **Smart and adaptable** Software Engineering. The future of development is not in actual physical coding but in structured design. This will utilize:

- Battle tested **design** and **architectural patterns**..
- UML **visualization**.
- New age **AI code generation** (for both tests and actual working code).
- **AI based log analysis** with various analytics, to assess the health of your code .

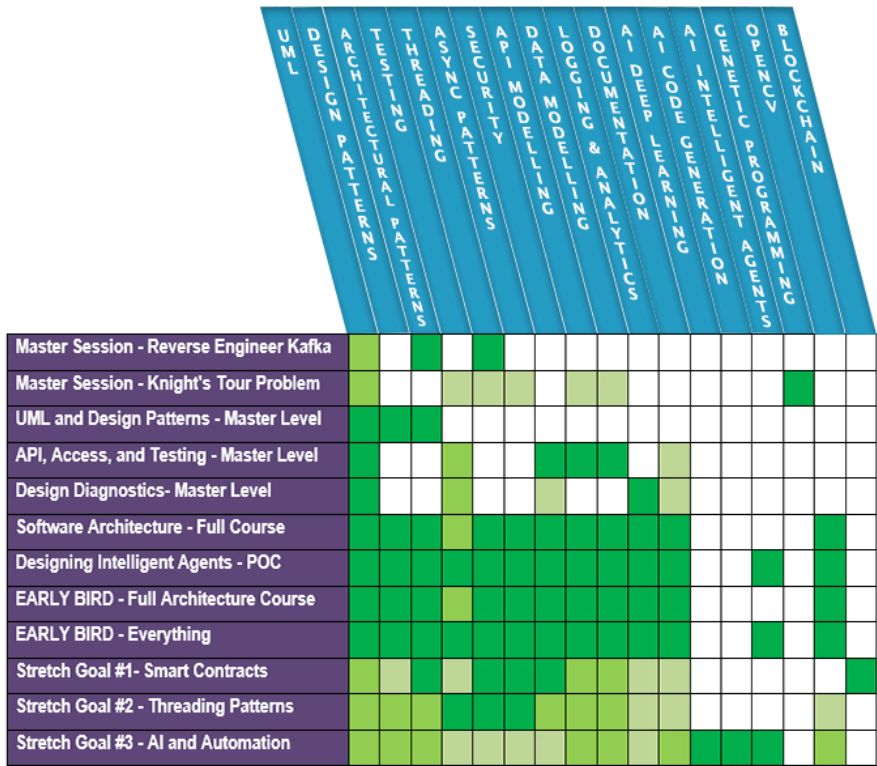
The world **will need** more Software Architects who can design applications and systems in a reliable and scalable fashion. As such, the opportunities for **working on big projects** and **making more money** will be open to those who can do that.

Do not forget, **code is the incarnation of a vision**. It is the final expression of a desire to create a working project. How you **plan, document, and visualize** your effort, makes all the difference.

The aim of design is to focus on the **big picture** first, and progressively fill in the details later.

This course will teach you that and more.

Are You Ready?



THE HEART OF WHAT SOFTWARE ARCHITECTURE IS

Let's try a simple challenge:

- Go to GitHub
- Grab a random project of medium size written in java (or some other programming language. Specific language used is not relevant to this exercise.)
- Download and open the code.
- Can you tell right away what the APIs are?
- Can you tell what the APIs are doing?
- Can you tell how the different layers of code communicate?
- Can you tell which parts of the API are thread-safe?
- Can you tell how to modify the behavior of the code just from looking at it? How easy is it to modify the code?
- Can you tell anything about the threading behavior of the application? Like how many concurrent threads will be running as part of the run-time execution?
- Can you tell how secure the application is? is it secure at all?

The most probable answer to most of the above questions will be a **"No."**

```
108
109     let stageDocs: Document[];
110     for (const linkedDoc of doc.linkedDocuments) {
111         //
112         // place the linked doc into the map based on its group type name
113         if (this.traceabilityMap.has(linkedDoc.groupTypeName)) {
114             stageDocs = this.traceabilityMap.get(linkedDoc.groupTypeName);
115         } else {
116             stageDocs = new Array<Document>();
117         }
118         stageDocs.push(linkedDoc);
119         console.log('Linked Docs Array: ' + JSON.stringify(stageDocs));
120         this.traceabilityMap.set(linkedDoc.groupTypeName, stageDocs);
121     }
122 }
```

Code can be Hard to understand without an Architecture.

Most projects you find on GitHub (or other repositories) will not come with an architecture or any other design document, so you would have to roll up your sleeves, dig in your heels, get a 12-pack of Jolt-Cola, and peer deep into the project code to understand **how** it works. And you are in no way guaranteed that there is any reasonable standard to follow.

```

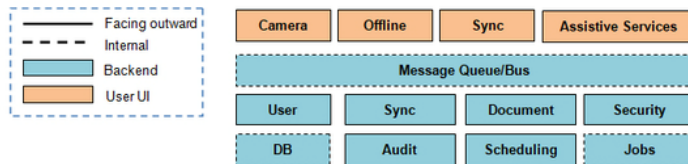
319 private Quadrilateral getQuadrilateral( ArrayList<MatOfPoint> contours , Size srcSize ) {
320     double ratio = srcSize.height / scale_ratio;
321     int height = Double.valueOf(srcSize.height / ratio).intValue();
322     int width = Double.valueOf(srcSize.width / ratio).intValue();
323     Size size = new Size(width,height);
324
325     for ( MatOfPoint c: contours ) {
326         MatOfPoint2f c2f = new MatOfPoint2f(c.toArray());
327         double peri = Imgproc.arcLength(c2f, doided: true);
328         MatOfPoint2f approx = new MatOfPoint2f();
329         Imgproc.approxPolyDP(c2f, approx, epsilon: 0.02 * peri, doided: true);
330
331         Point[] points = approx.toArray();
332         // select biggest 4 angles polygon
333         if (points.length == 4) {
334             Point[] foundPoints = sortPoints(points);
335             if (insideArea(foundPoints, size)) {
336                 return new Quadrilateral( c , foundPoints );
337             }
338         }
339     }
340     return null;
341 }

```

Highly Specialized Code can be hard to understand

See, understanding a project at the level of code is hard. And it should be completely unnecessary.

When there is no design you incur a **debt**. A **debt of time**. You will be wasting a lot of time trying to understand even your own code if you did not properly design and document it. And believe us, this is what happens to most projects... they eventually create ugly and unreliable code. Unless... the projects are properly engineered and documented from the start, with an easy-to-follow breakdown of modules.



Simple Example of a High-Level Module Diagram

So, let's go back to what is at the heart of Software Architecture: **Design**.

Design is powerful.

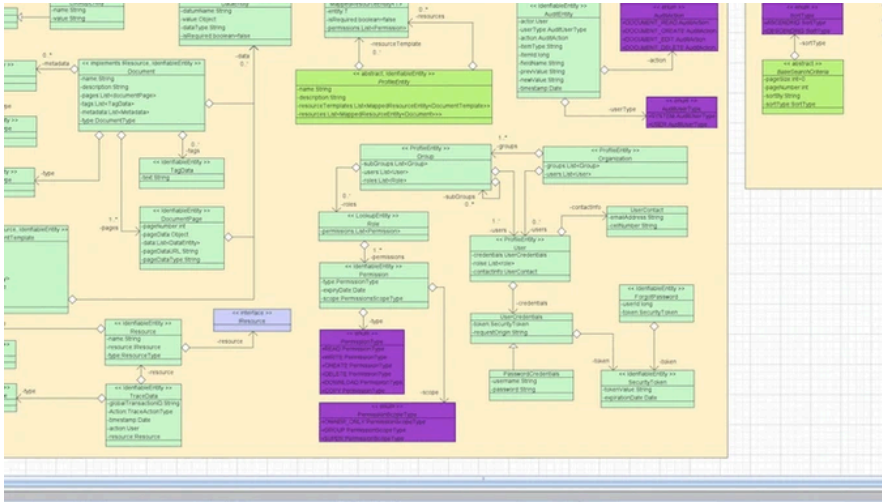
Why?

- Because it follows a standard.
- Because it allows you to manage project complexity in bite-size (i.e. manageable) chunks and provides you and your team with a proper **visualization** of your code and your design patterns.
- More importantly, it allows you to use a version of the **scientific method** to test your design before it even becomes code. This saves tremendous amount of time and effort.
- Good software design saves your organization time and money and allows you to create projects that otherwise would not be tangible or

viable. It also fosters good **documentation**.

- And, even more importantly, it standardizes the overall approach to your project and enables code **re-usability**.

So, **design** is powerful because **re-usability** is powerful and because **standardization** is powerful. Good Design creates a **high level of visualization**, predictability, and assurance.



Bird's Eye View Of a Design is Super Helpful

Great design creates **beautiful code**.

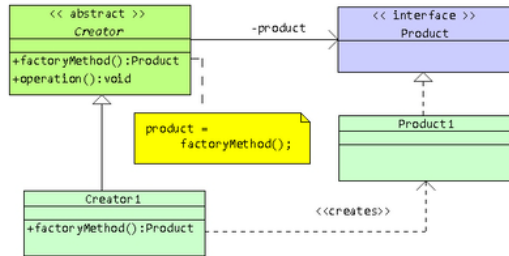
THE **WHAT** OF SOFTWARE ARCHITECTURE

Architecture is a blueprint.

It is a high level visualization that follows set standards. It is a **blueprint** for structure and behavior. It captures a number of **high-level** aspects of design such as :

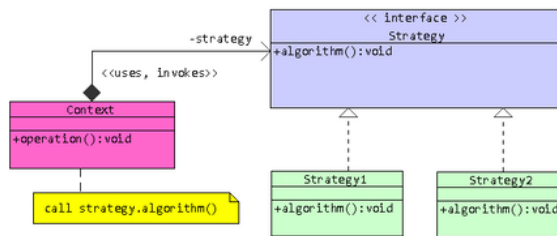
- Is the API **thread-safe**?
- Is the data **mutable**?
- What **design patterns** are used?
- What are the **pre-conditions** and **post-conditions** for the API interfaces?
- Is the behavior properly **encapsulated** and reliable?
- How does the software **recover** when there are errors?
- and more...

Software Architecture represents the process of designing and using a blueprint for your project. Having such a **blueprint** is a **huge advantage** as it allows you to get a **bird's-eye view** of your code and address any issues and challenges quickly and efficiently. It also creates a roadmap for maintenance and expandability. Design Patterns are the building blocks for such a blueprint.



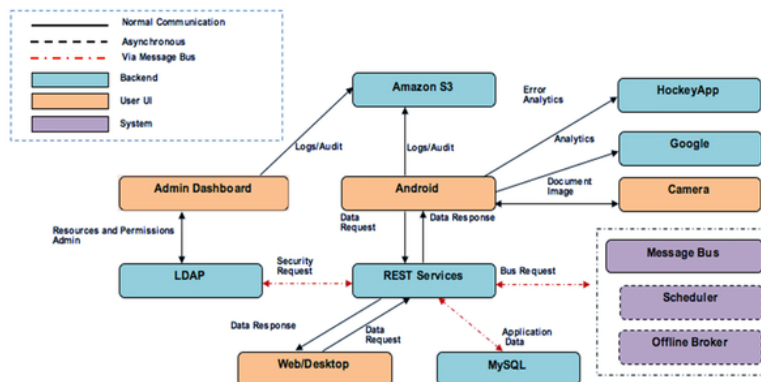
Factory Method - Example of a powerful yet simple Design Pattern

This course will teach you how to create such blueprints in the fastest and most efficient manner by utilizing the most powerful and useful design patterns.



Strategy Pattern - One of the most powerful design patterns.

We will make a clear distinction between **Architectural Patterns** and more low-level **Design Patterns**.



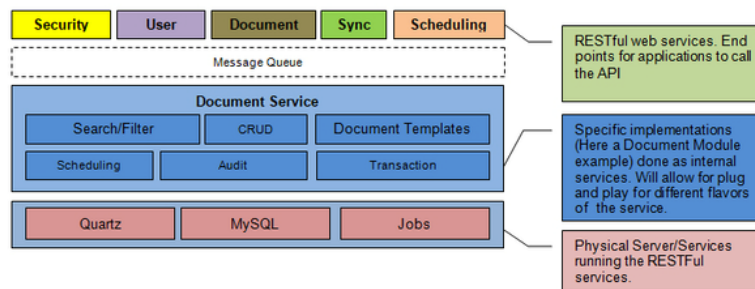
THE **WHY** OF SOFTWARE ARCHITECTURE

Why do you need to learn Software Architecture? For one thing it might earn you **more money**. Of the software developers out there, only a small fraction are actually Software Engineers so you will be more competitive.

But it is much more profound than making more money. You cannot create complex and flexible projects without good design. Any project will eventually hit a wall of complexity that cannot be surmounted, especially when many developers are involved and you need to make quick changes to code or even just try to debug all of it.

Code-level management **does not scale** well because it is done at the localized level of code itself. This usually leads to a lot of ugly code and questionable workarounds. Design on the other hand **does scale**, because it is external to the code.

Design promotes global concepts of **encapsulation** and **delegation of responsibilities**.



Layering and Modular Encapsulation - One of the facets of Software Architecture

Think of it this way. There is a reason why modern video games are written around **game engines**, and back-end database code is written around such standards as **JDBC**. Writing reusable and standardized code pays off greatly. **Architecting reusable code** is at the heart of this.

WHY SHOULD **YOU** TAKE THIS COURSE?

Because design is powerful. This is worth repeating.

Given that there are only a finite number of hours in a day, it follows that you can only produce a certain amount of usable code per day. So you want to make sure that your efforts **do not produce wasteful and hard to maintain code.**

This course will **teach you** just that.

The main point though is this: we will **cut out** the fluff and the **unnecessary stuff** and present you with the most proven and useful elements.

We will be using the following **methods** in our teaching:

- **Video Lectures** - which will show you step by step what we are doing and how we are solving specific issues. We want to create a set of highly professional videos that will explain in the **simplest yet profound** manner the most important elements of how to become a successful Software Architect.
- **Written Materials** - which will be a number of document templates as well as cheat sheets, diagrams, and PDF version of the Video Lectures. You will be able to use the templates in your own projects.
- **Interactive message boards** - which will allow us to clarify any issues and provide you with feedback on your progress.
- **Progressive Approach** - which will immerse you in a set of progressive and practical exercises so this way you will learn everything by doing it. The knowledge we want to teach you is both practical and theoretical. We want to make sure that you master both.

Knowledge is power, and with this in mind the main goals of the course materials are:

1. To allow you to discover the great joy that comes from successfully designing working projects. **Nothing succeeds like success.**
2. We want to create a progressive learning experience. It is all about the layout of topics, the type of exercises, and enabling you to successfully create working projects through design.
3. The aim is to elevate the process of planning and testing to the level of **intuitive endeavor.**

WHAT WILL **THIS** COURSE DO FOR **YOU**?

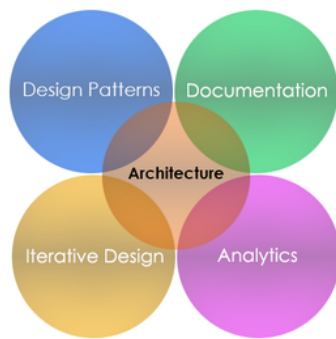
This course will teach you all the **practical elements** of Software Engineering.

There is a lot of stuff out there that does not really work or is very rarely used.

Based on our real-world experience working for companies such as **NASA**, **Google**, **Facebook**, **Microsoft** (among others) we will teach you what you need to know in a focused and pinpointed manner.

We will teach you how to **reverse-engineer** existing code as well as how to **forward-engineer** new code. We will show you how to visualize already existing code (like from GitHub) and build new code by visualizing the operations, interactions, and behavior of your new functionality.

What you need to know can be encapsulated into what we call the **Four Cornerstones** of a successful architecture.



Four Cornerstones of Successful Software Architecture Approach

The Four Cornerstones of Software Architecture:

- Design Patterns
- Documentation
- Iterative Design
- Analytics

Design Patterns - You will learn the most powerful and useful design and architectural patterns. This is at the heart of proper encapsulation.

Documentation - We will teach you how to document and visualize your architecture when making it into a true blueprint. Keeping well presented

and up-to-date documentation is crucial. We will provide you with all the templates you need.

Iterative Design - We will teach you how to start with a skeleton design and progress from design, through a POC (proof of concept,) to a finished product. We will write test suites to make sure that we do not introduce side-effects as we develop the modules. We will also add security to the solution.

Analytics - We will design and create pluggable logging, auditing, and analytics modules to keep track of the health of our application, including security logs and audits, as well as AI based predictive analytics to anticipate failures.

In addition we will also illustrate a number of technologies as part of our projects such as:

- Deep Learning
- Computer Vision
- Genetic Programming
- Cryptocurrencies (such as Ethereum and Bitcoin) and Blockchain.
- Encryption/Decryption

We will also look into self generating code.

But all in all we will keep the course material focused, experience-forming (through many exercises,) and relevant.

HOW WILL **THIS** COURSE BE STRUCTURED?

This course will be first and foremost driven by **practical** examples and exercises.



It will be broken into a number of modules. You can pick and choose which modules fit your needs but of course we would urge you to take the full course module!

Module 1: Master Session - Reverse Engineer Kafka

This is a stand-alone session that showcases architecture of the Kafka Server. This is a great way to see how UML and design patterns are used in a highly successful product. This is also a great session for understating threading. You will:

- Reverse Engineer The Kafka Messaging Server
- Study the Kafka UML. We will explore the Class and Sequence diagrams.
- Study the Kafka threading model.
- Go through the Kafka Architectural Design.

Module 2: Master Session - Knight's Tour Problem

This is a stand-alone session that showcases the abstraction of an algorithmic solution for the Knight's Tour chess problem. In this session we will look at how to architect and design the specific layers of the application (including data storage, algorithm, and UI layers)

We will look at pluggable algorithms including a Brute Force approach as well as Genetic Programming solution. We will have both a web and Android based solution for the UI. You will:

- Explore the MVC pattern
- Explore multiple ways of solving the tour problem including Brute Force.
- Explore Genetic Programming solution for the tour.

Module 3: UML and Design Patterns - Master Level

This is the **most important module** as it is the foundation for Architectural thinking. You will be exposed to UML as well as the most important design and architectural patterns. We will go through many exercises so that UML becomes second nature to you! We also want you to start thinking in terms of design patterns as one would in terms of Lego pieces. You will:

- Learn Class, Use-Case, and Sequence Diagrams
- Practice makes perfect! Learn through many exercises.
- Learn about Asynchronous concerns.
- Study Complex Class and Sequence Diagrams
- Study Asynchronous Modelling and different threading protocols
- Master the most important Design patterns. We will use the most useful and proven design patterns in code and UML. Learn how to use the patterns when designing an architecture.
- Master Architectural Patterns. We will use most useful and proven Architectural patterns including working with a **messaging bus**.

- Investigate how to design self-generating code.

As part of this module we will Design Conway's Game of Life and create it as an application for both Web and a mobile device.

Module 4: API, Access, and Testing - Master Level

This module concentrates on how to design APIs and data elements for your architecture. This will also talk about how to secure your API and your application, as well as how to design and write test suites for your application. You will:

- Learn how to create self-initialized and self-diagnosing data modules.
- Master API formulation and implementation.
- Learn how to create reusable APIs
- Add "smart" beans (data elements) to your architecture. Design the data to be smart, adaptable, and self-diagnosing.
- Learn as to why write test code?
- How to design and Create Test Suites.
- Learn what to cover when testing.
- Learn the most efficient way to write tests.
- Learn how to create a code test generator.
- Explore how to test and isolate any unwanted side effects when maintaining code.
- Go through practical tools for testing.

Module 5: - Design Diagnostics- Master Level

This module concentrates on how to design logging, auditing and analytics for your project through an architectural approach. You will:

- Explore Logging - What to log and when to log it.
- Learn about using AOP.
- Explore Auditing - How to create auditing modules for your architecture.
- Learn about Analytics - How to analyze analytics data about how your code is.
- Learn how to design reusable log modules.
- Add smart logging and analytics to your architecture code. Learn how to properly read and analyze your logs. Add user-gesture auditing and analytics to your code as a plug-in.

Module 6: - Software Architecture - Full Course

This module ties all the previous modules together. Here you will look at 5 projects and you will learn about the full documentation cycle to document

your architecture using specific templates for different planning and design stages. You will:

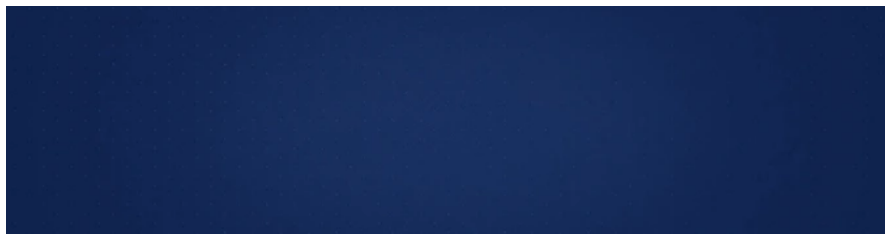
- Learn the Golden Rules of Proper Documentation.
- Explore SOW, Specification, Architecture, and Technology Stack templates.
- Learn how to keep all documentation in Sync as code evolves.
- Architect an Uber clone.
- Architect Instagram Clone.
- Architect Secure WhatsApp clone.
- Architect an OpenCV Document detector with OCR.
- Architect a An Audio recorder application that will allow for tagging of audio in documents as well as search through audio data for keywords.

Module 7: Designing Intelligent Agents - POC

We will look into designing protocols for intelligent Agents. This will be an introduction to Intelligent Agent branch of AI and will be all about designing protocols for Intelligent Agent communication.

The fun part about this module is that it will be driven by a POC for different Agent Protocols. We will look at ways of creating protocols that could for example make decisions such as best time to slot an appointment for you with a dentist, without your intervention. We will learn here how to create rules, goals, and behavioral modules. You will:

- Explore and learn how to design learning modules.
- Define and learn how to encode goals and rules for the AI Agents to follow.
- Learn about and define adaptable protocols.



Stretch Goal #1 - Smart Contracts

Create a reusable architecture around **smart contracts** utilizing **blockchain**. Learn all about the intricacies of blockchain and **cryptocurrencies**.

- Smart Contracts - Look at smart contracts from an architectural perspective. When are they appropriate and when they are not.

- Reverse engineer a number of **cryptocurrencies** (Ethereum, Bitcoin)

BONUS - TopCoder experience podcast.

- Additionally we will stream a podcast about our experience with TopCoder and will try to share with you insights about how we were able to compete within and learn from an amazing community.
- In fact this will be an introduction to the amazing TopCoder community through the optics of our years of competitions.
- We hope to give you advice about what to do to be a successful TopCoder!

Stretch Goal #2 - Threading Patterns

Proper threading and **thread control** is crucial in today's applications. Design adaptable threaded architectures with complete control over different thread models. Control timeouts, resource caps etc...

- Think of **game engines**. They need very smart and adaptable threading architectures. In this stretch goal we will look to create a highly adaptable threading architectures for game and other engines. We will look at creating an asynchronous Architecture with an adaptable pattern for thread control.
- Learn how to create self-adapting and self-repairing modules using daemon threads to watch over the health of the application.
- Using the thread engine we will develop a rudimentary **particle physics engine**.

Stretch Goal #3 - AI and Automation

Harness AI to help with code generation as well as diagnosis of live code. Learn how to incorporate **deep learning** and **agent technologies** into your architectures.

- AI and automation - How can AI be utilized and added to architectural patterns
- Smart code generation. AI writing self-generating code - Looking into how AI can be used to diagnose and "heal" issues in data and code, in real-time
- Logging and Analytics - How can AI be used to alert us about code issues before they happen?
- AI and audits - Can AI be used to alert us about security and behavioral issues?

Summary of what we will offer:

- Mastering the most useful Design and Architectural patterns.

- Learning UML and design visualization. (this will take your from a novice to a master in modeling) .
- Implementing State initialization (how to configure and initialize your projects)
- Testing your code with Architectural rigor.
- Learning best practices for Data Modeling
- Designing around Thread-Safety
- **Logging** and **Auditing** as main ways of working with project behavior analytics. **Utilization of AI** in log analysis.
- Designing Technology Stack and its considerations.
- We will reverse-engineer a number of popular projects (such as Ethereum code base) to give us a good feel for visualizing large projects using UML.
- We will work on a **number of hands-on projects** and we will design our solutions while mastering, in a practical way, the Architectural paradigms. We will design the solutions in a manner that will follow a well tested workflow and we will produce the necessary artifacts such as the following:
- **SOW** (Statement of Work) which will define the scope of what we will try to achieve.
- **Application specification** which will outline and breakdown the functionality that we will be implementing. This will be UX agnostic.
- **Technology Stack** analysis.
- **Data Modeling** document (if necessary).
- **Intelligence Modeling** (for any AI based solutions)
- **UML visualization** of the solution.
- **Test Suite design** for testing of both behavior as well as speed and latency of the solution.
- **Proof of Concept.**
- Proper **logging**, **auditing**, and **code analytics** design.
- **Security specification.**

STRETCH GOALS

STRETCH GOAL #1

\$10K **SMART CONTRACTS**
BONUS - TopCoder Experience Podcast



Create a reusable architecture around SMART CONTRACTS utilizing BLOCKCHAIN. Learn all about the intricacies of BLOCKCHAIN and CRYPTO-CURRENCIES.

STRETCH GOAL #2

\$20K **THREADING PATTERNS**



Proper threading and THREAD CONTROL is crucial in today's applications. Design adaptable threaded architectures with complete control over different thread models. Control timeouts, resource caps etc...

STRETCH GOAL #3

\$30K **AI AND AUTOMATION**



We will harness AI to help us with code writing as well as diagnosis of live code. We will tackle how to incorporate DEEP LEARNING and AGENT TECHNOLOGIES into our architectures.

Stretch Goals

WHAT TOOLS WILL **WE** USE IN THIS COURSE?

We will present a number of Architectural and Software Engineering principles using specific languages for illustrative purposes. We are not gearing this course to any given programming language but we will be using multiple language examples (such as Java, C#, Python, or JavaScript) to make our points and examples clear. The main **emphasis** though, will **always be** on **design**.



- To make the course outcome as practical as possible we will also provide samples of code that could be run using the following tools:

- We will be working with projects that will touch upon the following technologies:

- We will also access some projects in GitHub to illustrate **reverse-engineering** code sources into **visualized architectures**.

19/25

"*Teach to the Problem not the tool*" has at its heart the aim of teaching problem solving and not merely learning specifics of tools.



As an example consider learning the Java programming language with its functions, loops, if statements etc... if you just learn how to use the language without understanding the deeper aspects of the **Object Oriented Paradigm** then you will never really master the full potential of Java (or any other language for that matter)

Once you understand what OOP tries to solve then the tool (i.e. Java) becomes to a great extent self-evident.

In this course we are taking the approach of **teaching you problem solving** as our prime directive.

WHAT MAKES **THIS** COURSE **SPECIAL**?

YOU make this course special.

It is always about the students and we are passionate about sharing with you our knowledge and experience.

We will strive to make this course **fun** and **instructive**, as well as **cutting edge**.

We hope that our combined years of experience in both competitive development and design with **TopCoder** as well as targeted development for **NASA**, **Google**, **Facebook**, **IBM** and others, makes us uniquely qualified to impart on you the best of what we have learned.

We also bring to the table academic credentials in research as well as long-standing experience in teaching of computer science related topics.

YOUR INSTRUCTORS: **ABOUT US**

We come from a deep **crowdsourcing** community of developers. We are [TopCoder](#) veterans as well as [TCO Hall of Famers](#) (TCO - TopCoder Open - a worldwide yearly development and design competition). [TopCoder](#) has more than **1 million competitive members**.



Piotr Paweska

(TopCoder handle AleaActaEst)

Computer Science BSc (University of Toronto)

Emphasis on Software Engineering, Algorithmic Computation (such as Genetic Programming) and AI

Years of experience in Teaching Computer Science courses as well as Practical Development courses

Years of competitive Development with TopCoder

- TCO Specification Champion (2009)
- TCO Component Design finalist for years 2014, 2013, 2010, 2008, 2007, 2006
- TCO Architecture Finalist (2009)



Michael Paweska

(TopCoder handle argolite)

Physics BSc (University of Vancouver)

Years of competitive Development with TopCoder

- Two time TCO Design Champion (2012, 2010)
- TCO Component Design Finalist for years (2014, 2013, 2009, 2008)
- TCO Architecture Finalist (2009)

Risks and challenges

Any ambitious endeavor carries a number of risks and challenges with it. And so it is with creating a full fledged online course.

The main challenge for us is to create a way to teach the students our experience. We want to show the beauty of the design process and how powerful the knowledge of design is but at the same time we want to remove the many obstacles and challenges that we have encountered in our own experiences.

"Knowledge is Power" - is a famous quote and nothing could be more true about the knowledge that goes into Software Engineering. We are eager and passionate about imparting such knowledge onto our students and seeing them acquire the power of Software Engineering approach.

It is our belief that armed with this knowledge the student will be on their way to become a Software Architect.

We will be using the following methods:

1. Video Lectures - which will show the students step by step what we are doing and how we are solving specific issues. We want to create a set of highly professional videos that will explain in the simplest yet profound manner the most important elements of how to become a successful Software Architect.

2. Written Materials - which will be a number of document templates as well as cheat sheets, diagrams, and PDF version of the Video Lectures.

3. Interactive message boards - which will allow us to clarify any issues and provide feedback to our students.

4. Progressive Approach - which will immerse the student in a set of progressive and practical exercises so they learn everything by doing it. The knowledge we want to teach the student is both practical and theoretical. We want to make sure that the student masters both.

We are prepared to spend the necessary hours, days, and weeks to bring the best content possible, on time.

There are many roads that lead to Rome but we would like to show you the shortest and at the same time the most scenic one.

[Learn about accountability on Kickstarter](#)

Questions about this project? [Check out the FAQ](#)

[Report this project to Kickstarter](#)

Similar projects to check out

See more



 Save our oceans. Take Wildtype mainstream. 
Justin Kolbeck



 Zen Tea Brew Infusions: Discover Delicious Rare Shizuoka Tea 
Zen Tea Brew
 11 days left • 629% funded



 **Faire in the Field: Year Two!**
Patrick Munroe-Smith



 **Laughing Gull Chocolates Take Flight**
Lindsay



 10 days left • 81% funded



Discover

Arts

Comics & Illustration

Design & Tech

Film

Food & Craft

Games

Music

Publishing

About

About us

Our charter

Press

Jobs

Support

Help Center

Our Rules

Creator Resources

Forward Funds

Brand assets

More

Newsletters

Kickstarter Project Updates

The Creative Independent

Mobile apps

Research

KICKSTARTER

English ▾

\$ US Dollar (USD) ▾



Kickstarter, PBC © 2026

[Trust & Safety](#) [Terms of Use](#) [Privacy Policy](#) [Cookie Policy](#) [Cookie Preferences](#) [Accessibility Statement](#)

[CA Notice of Consent](#)